

Monocular Visual SLAM Using Custom Video Input

Ma Qihao
Southern Illinois University Edwardsville
ME/ECE/CS 462 – Robotic Vision

April 2025

Contents

1	Introduction	2
2	SLAM Example Using the TUM Dataset	2
2.1	Overview of the Example	2
2.2	Key Results	2
2.3	Discussion	3
3	SLAM Using Custom Video Input	4
3.1	Preprocessing Pipeline	4
3.2	Map Initialization and Feature Matching	4
3.3	Feature Point Detection	4
3.4	Trajectory and Point Cloud Reconstruction	4
3.5	Discussion	5
3.6	Camera Calibration	5
4	Reflection and Future Work	6
5	Project Files	7

1. Introduction

Simultaneous Localization and Mapping (SLAM) is a foundational technique for autonomous systems to understand and navigate unknown environments. SLAM enables a robot to build a map of an unknown environment while estimating its pose within that map. Visual SLAM (vSLAM), in particular, uses a camera as the primary sensor. Monocular vSLAM uses a single camera, which makes it lightweight and cost-effective but presents unique challenges like scale ambiguity and depth estimation.

This project explores the implementation of monocular vSLAM using MATLAB. The work is divided into two parts: replicating a benchmark example using the TUM RGB-D dataset and implementing vSLAM on a custom-recorded indoor scene. The project involves feature detection, camera trajectory estimation, map reconstruction, and performance evaluation.

2. SLAM Example Using the TUM Dataset

2.1. Overview of the Example

The MathWorks example uses ORB-SLAM principles to implement monocular visual SLAM. The dataset is from the TUM RGB-D benchmark and provides pre-recorded images and ground truth trajectories. The core components of the example are:

- Reading a sequence of RGB images
- Detecting and extracting ORB (Oriented FAST and Rotated BRIEF) features
- Matching features between consecutive frames
- Estimating relative camera poses using matched features
- Triangulating 3D landmarks
- Performing loop closure and global pose graph optimization to refine the trajectory

The implementation leverages MATLAB's built-in visual SLAM framework, making it suitable for rapid development and testing.

2.2. Key Results

As shown in Fig. 1, the estimated camera trajectory closely follows the ground truth, with small translation errors over time.

Figure 2 illustrates the initial feature correspondences between frames, which help determine relative camera pose.

The resulting map points and estimated trajectories are visualized in Fig. 3, with loop closure corrections applied.

Detected ORB features across the scene are shown in Fig. 4, contributing to map triangulation.

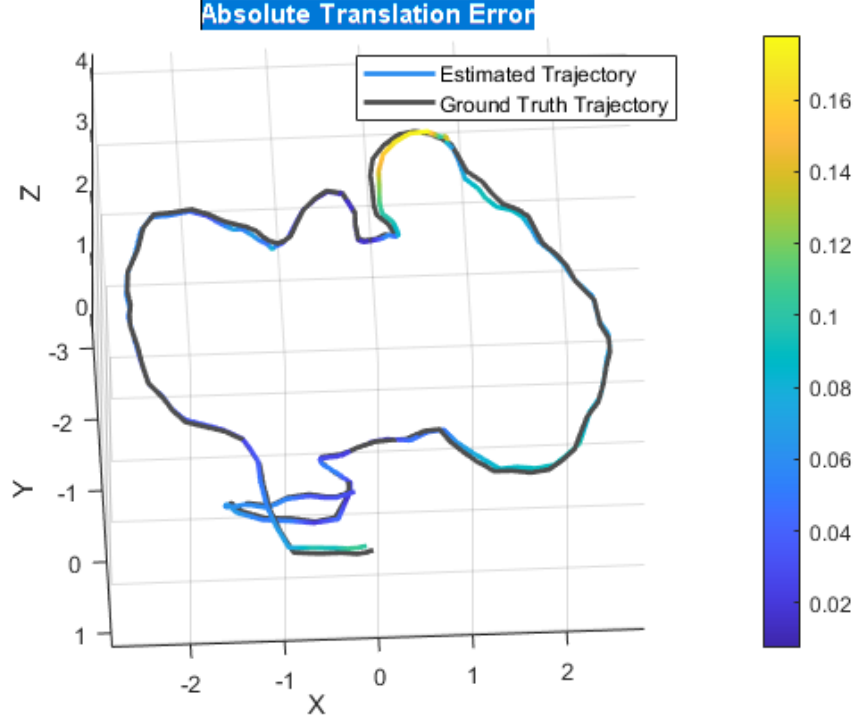


Figure 1: Absolute Translation Error: Estimated vs Ground Truth Trajectory

2.3. Discussion

These visualizations demonstrate the effectiveness of ORB-based SLAM in reconstructing a scene and estimating the camera trajectory. In Figure 1, the estimated trajectory aligns closely with the ground truth, validating the accuracy of the pose estimation. Figures 2 and 4 show the robustness of the ORB feature detector and descriptor in complex scenes. Figure 3 illustrates the sparse 3D map and the trajectory before and after optimization, highlighting the impact of loop closure.

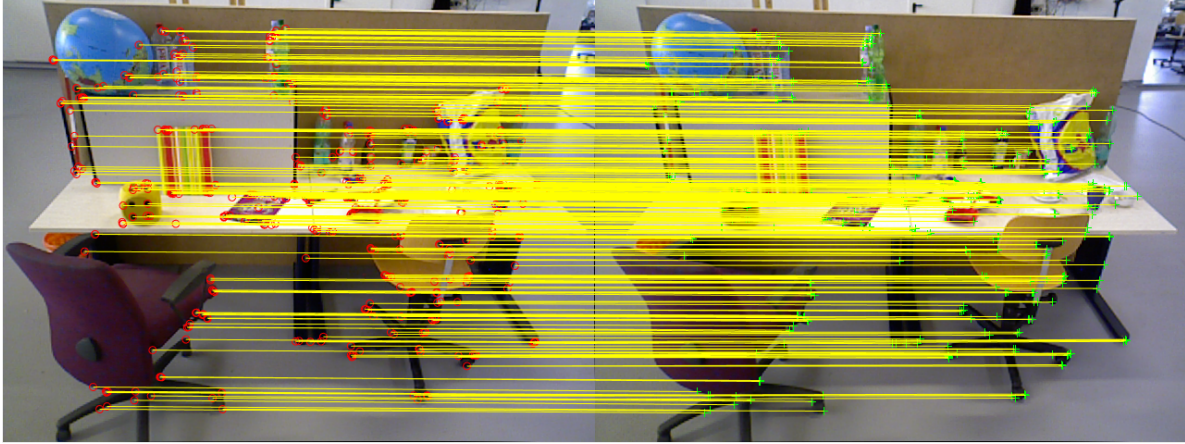


Figure 2: Feature Matching Between Keyframes During Initialization

3. SLAM Using Custom Video Input

3.1. Preprocessing Pipeline

A custom video (VIDEO04.mp4) was recorded in a personal indoor workspace. To prepare the data for SLAM, the following preprocessing steps were performed:

- Read frames using MATLAB's `VideoReader`
- Convert each frame to grayscale using `im2gray`
- Resize the frame to a fixed resolution of 1080×1920 pixels
- Store frames in a cell array for sequential access during SLAM processing

These steps ensure compatibility with MATLAB's visual SLAM tools and maintain uniform image dimensions across the dataset.

3.2. Map Initialization and Feature Matching

As shown in Fig. 5, initial frames are used to extract and match features to establish camera motion and triangulate initial 3D points.

3.3. Feature Point Detection

ORB feature detection is applied to detect robust and repeatable points in each frame, as seen in Fig. 6.

3.4. Trajectory and Point Cloud Reconstruction

The following figure (Fig. 7) displays the sparse 3D point cloud and the trajectory as estimated and optimized using bundle adjustment.

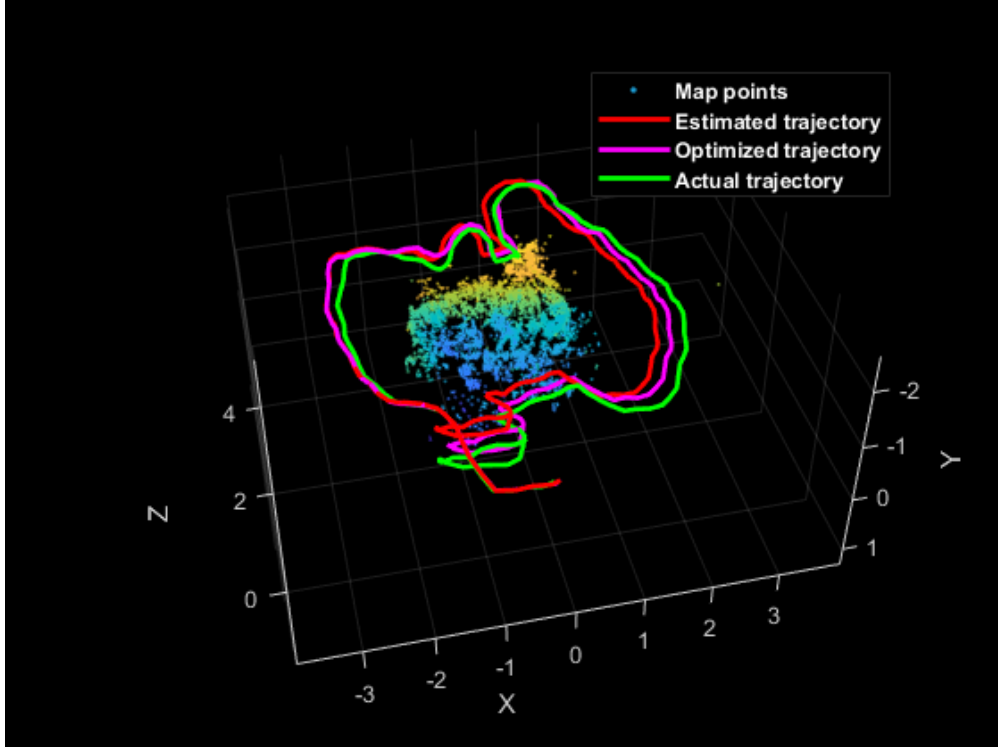


Figure 3: Map Points and Trajectory Visualization: Estimated, Optimized, and Ground Truth

3.5. Discussion

This custom experiment demonstrates the portability of the vSLAM pipeline to new environments. Despite lower lighting and less structured texture, the system was able to:

- Detect and track features across frames
- Estimate a consistent camera trajectory
- Reconstruct a sparse 3D map of the environment

The results (Figs. 5, 6, and 7) indicate the feasibility of using low-cost cameras for indoor mapping and tracking tasks.

3.6. Camera Calibration

Camera intrinsic parameters were measured using MATLAB’s Camera Calibrator App. Accurate calibration is essential for correct feature triangulation and camera pose estimation.

- Focal Length: [535.4, 539.2]
- Principal Point: [320.1, 247.6]
- Image Size: 1920×1080



Figure 4: Feature Points Used in Initial 3D Map Reconstruction

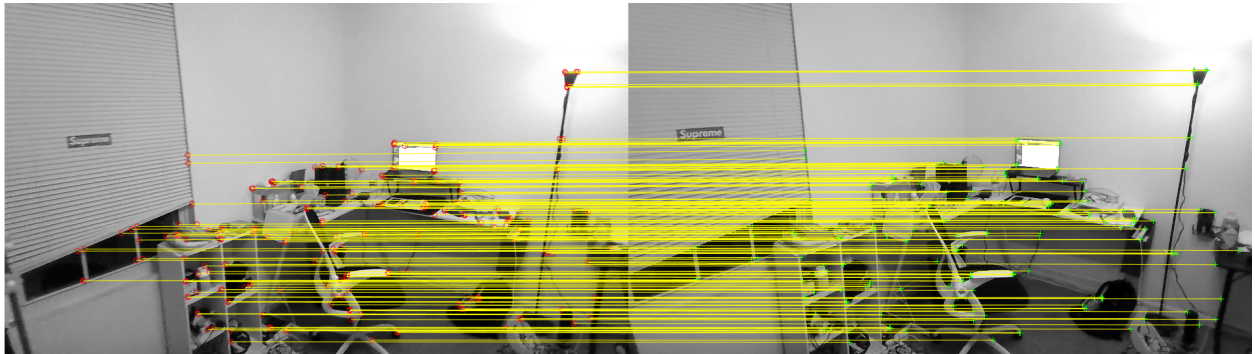


Figure 5: Feature Matching Between Initial Frames (Custom Video)

4. Reflection and Future Work

This project confirmed that monocular vSLAM can be effectively applied using both benchmark datasets and custom video inputs. Key lessons include the importance of camera calibration, frame quality, and feature matching robustness.

Future enhancements include:

- Integration of IMU data to improve motion estimation during fast movements
- Deployment of ORB-SLAM3 or OpenVSLAM for scale recovery and real-time performance
- Evaluating different feature descriptors such as SuperPoint or SIFT for increased accuracy



Figure 6: Feature Points Detected on a Frame from the Custom Video

5. Project Files

All datasets, MATLAB codes, result images, and videos are available at the following GitHub repository:

https://github.com/QiuhaoMMa/462_SLAM_project

References

- [1] MathWorks. Monocular Visual Simultaneous Localization and Mapping.
<https://www.mathworks.com/help/vision/ug/monocular-visual-simultaneous-localization-a.html>
- [2] TUM RGB-D Dataset.
<https://vision.in.tum.de/data/datasets/rgbd-dataset>

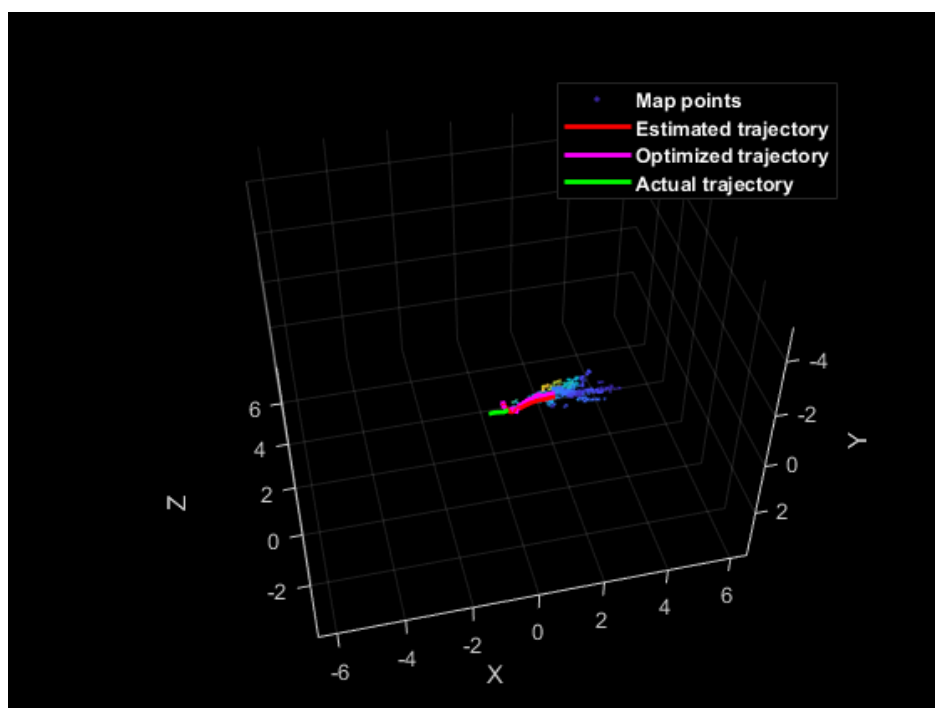


Figure 7: Map Points and Estimated, Optimized, and Ground Truth Camera Trajectories (Custom Video)